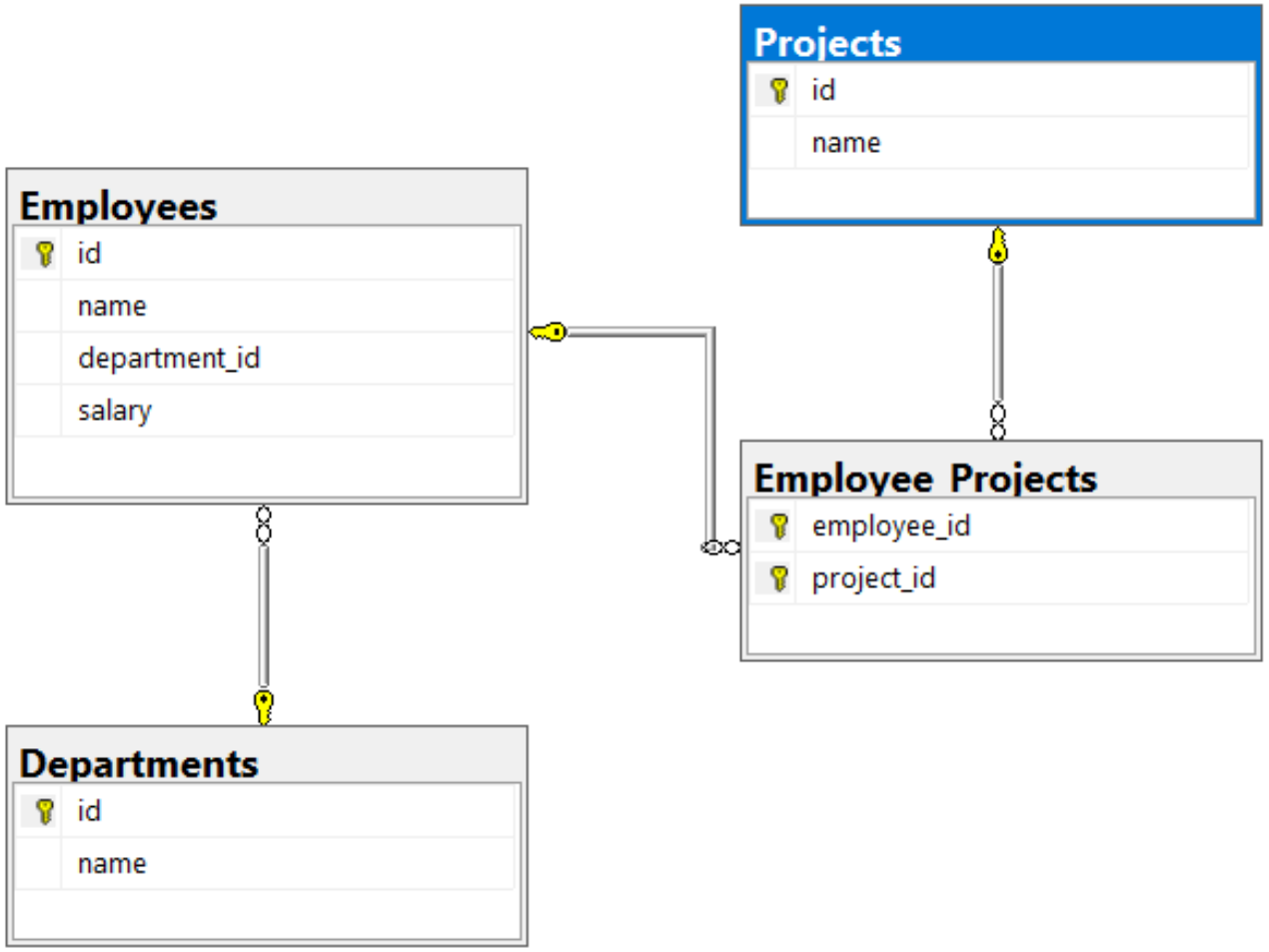
Secttion1

# Advanced Database

# Case Study


Projects
id
name
Employees
id
name
department_id
salary
Employee Projects
employee_id
project_id
Departments
id
name

# INSERT INTO

The **INSERT INTO** statement in SQL is used to add new records into a table

# UPDATE

- Used to modify existing data in a table. You can update the values of columns in specific rows based on certain conditions using the **WHERE** clause

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

# Employee t

| id | name | Department_id | salary |
|---|---|---|---|
| 1 | alice | null | 5000 |
| 2 | ali | null | 4500 |
| 3 | ahmed | 2 | 4800 |

# Difference Between DROP, DELETE, and TRUNCATE

# DROP TABLE

- DROP TABLE TableName;
- The table is permanently removed from the database.
- All data is lost forever.
- Any foreign key relationships referring to this table are also lost.

# DELETE FROM

- DELETE FROM TableName
- Deletes only the data while keeping the structure intact.
- Can be used to delete specific records using a WHERE condition.
- Can be rolled back (ROLLBACK) if executed inside a transaction.
- Does not reset the identity column (IDENTITY).

# TRUNCATE TABLE

TRUNCATE TABLE TableName;

Completely removes all data from the table.

Keeps the table structure intact.

Resets the identity column (IDENTITY) back to 1.

Cannot delete specific records (WHERE condition is not allowed).

```sql
create database employee_mangement_system;
Drop database employee_mangement_system;

---====--=====  CREATE TABLE ======--------------
CREATE TABLE Departments (
    id INT PRIMARY KEY IDENTITY(1,1),
    name NVARCHAR(100) NOT NULL
);

select *from Departments
drop table Employees
CREATE TABLE Employees (
    id INT PRIMARY KEY IDENTITY(1,1),
    name NVARCHAR(100) NOT NULL,
    salary DECIMAL(10,2),
        department_id INT,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON DELETE SET NULL,
);
CREATE TABLE Projects (
    id INT PRIMARY KEY IDENTITY(1,1),
    name NVARCHAR(100) NOT NULL
);
```

```sql
CREATE TABLE Employee_Projects (
    employee_id INT,
    project_id INT,
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES Employees(id) ON DELETE CASCADE,
    FOREIGN KEY (project_id) REFERENCES Projects(id) ON DELETE CASCADE
);
----------------===== INSERT INTO ======-------------
-- إدخال أقسام
INSERT INTO Departments (name) VALUES ('IT');
INSERT INTO Departments (name) VALUES ('HR');
INSERT INTO Departments (name) VALUES ('Acconting');
INSERT INTO Departments (name) VALUES ('Marketing');
INSERT INTO Departments (name) VALUES ('Finace');
-- إدخال موظفين
INSERT INTO Employees (name, department_id, salary) VALUES ('Alice', 1, 5000);
INSERT INTO Employees (name, department_id, salary) VALUES ('Bob', 2, 4500);
INSERT INTO Employees (name, department_id, salary) VALUES ('Charlie', 1, 6000);
INSERT INTO Employees (name, department_id, salary) VALUES ('mohamed', '', 6000);
-- إدخال مشاريع
INSERT INTO Projects (name) VALUES ('Project A');
INSERT INTO Projects (name) VALUES ('Project B');
```

```sql
-- ربط الموظفين بالمشاريع (Many-to-Many)
INSERT INTO Employee_Projects (employee_id, project_id) VALUES (1, 1);
INSERT INTO Employee_Projects (employee_id, project_id) VALUES (1, 2);
INSERT INTO Employee_Projects (employee_id, project_id) VALUES (2, 1);
INSERT INTO Employee_Projects (employee_id, project_id) VALUES (1, 1);
--------------======UPDATE =========
-- تحديث راتب الموظف Alice
UPDATE Employees
SET salary = 5500
WHERE name = 'Alice';

-- تحديث اسم القسم IT إلى "Information Technology"
UPDATE Departments
SET name = 'Information Technology'
WHERE name = 'IT';
---------------===== DELETE ================
-- حذف موظف معين (يحذف علاقاته من Employee_Projects تلقائيًا بسبب ON DELETE CASCADE)
DELETE FROM Employees WHERE name = 'Bob';
-- حذف مشروع معين
DELETE FROM Projects WHERE name = 'Project B';
```

Thanks!